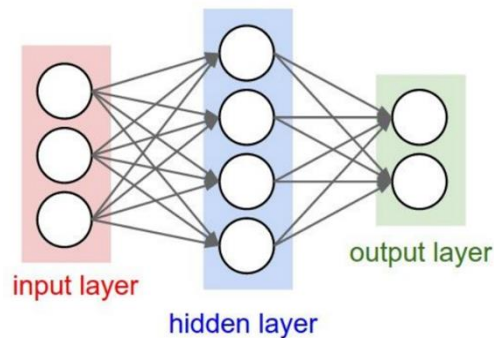


Atomic Units of Language

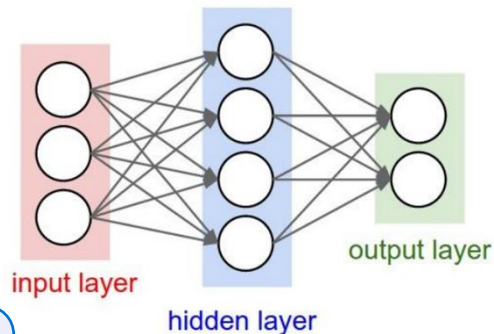
CSCI 601-471/671 (NLP: Self-Supervised Models)

Recap: Neural Nets

- A powerful function-approximation tool.
- Can be trained efficiently via Backpropagation.
- Our focus here: how to use NNs for language modeling.

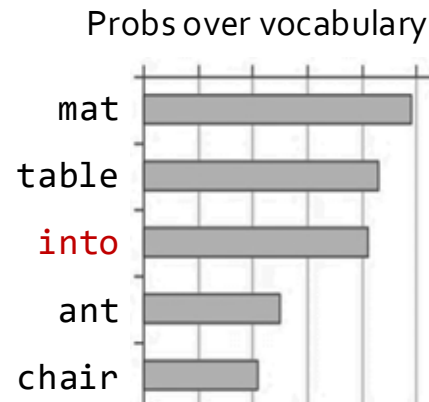
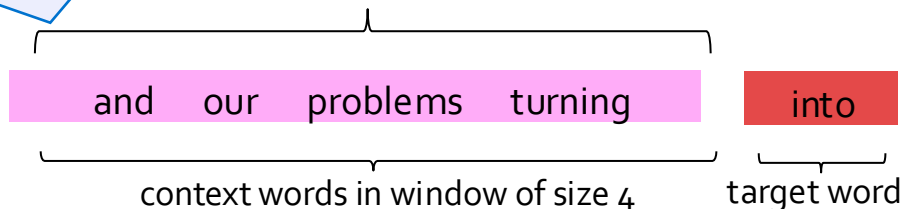


Big Picture: Language Modeling + NNs



Remember NNs expect numbers. How do you feed a string to neural net?

$$f(\mathbf{x}, \Theta)$$



Atomic Units of Language

The cat sat on the mat.

Which one should we use as **the atomic building blocks** for modeling language?



- words split based on white space?

BOS, The, cat, sat, on, the, mat., EOS

- characters?

BOS, T, h, e, SPACE, c, a, t, SPACE, s, ...

- bits??!

01010100 01101000 01100101 00100000
01100011 01100001 01110100 ...

Cost of Using Word Units

- What happens when we encounter a word at test time that **we've never seen in our training data**?
 - *Loquacious*: Tending to talk a great deal; talkative.
 - *Omnishambles*: A situation that has been mismanaged, due to blunders and miscalculations.
 - *COVID-19*: was unseen until 2020!
 - Acknowledgement: incorrect spelling of "Acknowledgement"
- What about related words?: "dog" vs "dogs"; "run" vs "running"
- We would need a **very large** vocabulary to capture common words in a language.
 - Very large vocabulary size makes training difficult
- What happens with words that we haven't seen before?
 - With **word level** tokenization, unseen words map to [UNK], **losing their identity**.
 - Also, not all languages have spaces between words like English!

Cost of Using Character Units

- What if we use **characters**?

- Pro:**

- (1) **small vocabulary**, just the number of unique characters in the training data.
- (2) fewer out-of-vocabulary tokens

- Cost:** **much longer input sequences**

- As we discussed, modeling long-range dependences is very challenging.
- Representing long sequences is computationally costly.

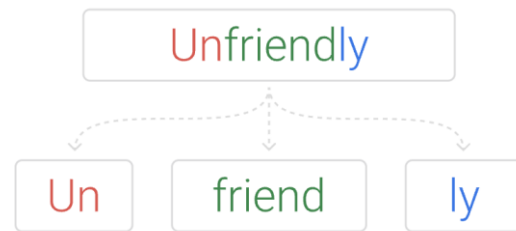
a	→	1
b	→	2
c	→	3
d	→	4
e	→	5
f	→	6
g	→	7
...	→	...
1	→	27
2	→	28
3	→	29
...	→	...
!	→	37
...	→	...
à	→	256

the	→	1
of	→	2
and	→	3
to	→	4
in	→	5
was	→	6
the	→	7
is	→	8
for	→	9
as	→	10
on	→	11
with	→	12
that	→	13
...	→	...

malapropism	→	170,000
-------------	---	---------

Subword Tokenization: A Middle Ground

- Learn reusable text pieces from a corpus; these *sometimes align* with morphemes
- Subword tokenization is the best of both worlds
 - Common words are preserved in the vocabulary
 - Less common words are broken down into sub-words
 - This handles the problem of unseen words and large vocabulary size
- Dominantly used in modern language models.



Trying an actual tokenizer

```
from transformers import AutoTokenizer

text = "The unbelievably fluffy cat purred."

for model in ["Qwen/Qwen3-0.6B", "deepseek-ai/DeepSeek-V3"]:
    tokenizer = AutoTokenizer.from_pretrained(model)

    ids = tokenizer.encode(text, add_special_tokens=False)
    tokens = tokenizer.convert_ids_to_tokens(ids)

    print(model)
    print(tokens)
    print(ids)
```

Trying an actual tokenizer

- Output for Qwen3: 8 tokens:

```
['The', 'Ġunbelie', 'vably', 'Ġfluffy', 'Ġcat', 'Ġp', 'urred', '.']  
[785, 38937, 88134, 67561, 8251, 281, 8130, 13]
```

- Output for DeepSeek-V3:

```
['The', 'Ġun', 'bel', 'iev', 'ably', 'Ġfluffy', 'Ġcat', 'Ġpur', 'red', '.']  
[671, 612, 12870, 41607, 3188, 81340, 7539, 2805, 1157, 16]
```

```
from transformers import AutoTokenizer  
  
text = "The unbelievably fluffy cat purred."  
  
for model in ["Qwen/Qwen3-0.6B", "deepseek-ai/DeepSeek-V3"]:  
    tokenizer = AutoTokenizer.from_pretrained(model)  
  
    ids = tokenizer.encode(text, add_special_tokens=False)  
    tokens = tokenizer.convert_ids_to_tokens(ids)  
  
    print(model)  
    print(tokens)  
    print(ids)
```

Trying OpenAI's Tokenizer

- How many tokens do we have here? (note, there are two spaces before the number.)
 - Where would you put the token boundaries?
What happens to the unfamiliar word, the spaces, and the number?

The cat saw a quokka 12345.

The cat saw a quokka 12345.

Frequent words are
one token

Rare word split into
multiple tokens

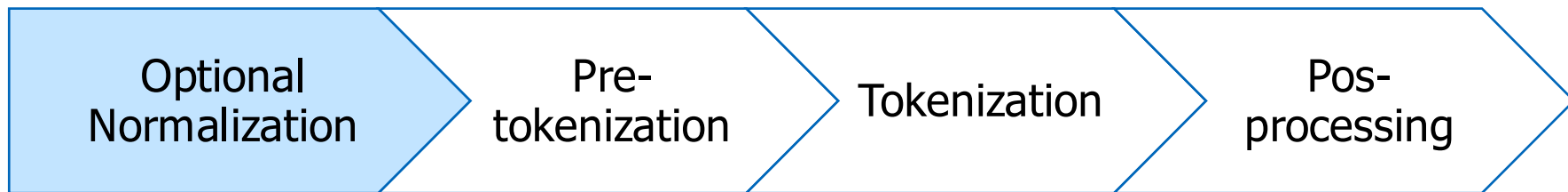
Large number split
into multiple tokens

Space before “cat” is part of
the token. So, the tokenizer
probably has two cat tokens:
w/ and w/o space.

Extra space
is counted
toward extra
tokens

<https://platform.openai.com/tokenizer>

The Tokenization Pipeline



- Converts text into a standard format to reduce variability.
 - Strip extra white spaces between words and sentences
 - Removing punctuations (“Hello, world!” → “Hello world”)
 - Unicode normalization (more on this in a sec),
 - ...

Unicode normalization

- There are many characters that look very similar (or identical) but are different in Unicode:

Roman "A"	A	U+0041
Fullwidth "A"	A	U+FF21

- There are various Normalization Forms to collapse visually/semantically-duplicate encodings (fullwidth vs ASCII, ligatures, old typographic variants) into one token.

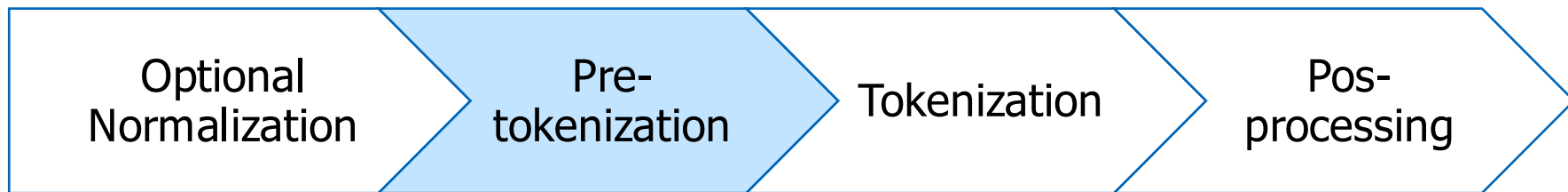
Positional variant forms	ε	→	ε
	ε	→	ε
	ε	→	ε
	ε	→	ε

Circled variants	①	→	1
Width variants	力	→	力
Fractions	¼	→	1/4

- NFKC normalization is a double-edged sword:
 - Benefit:** shrinks vocabs size (and more compact model)
 - Cost:** it's lossy and not reversible. After NFKC, we cannot tell whether A originally came from 'A' or 'A'.

See UAX #15 for more examples and details: <https://www.unicode.org/reports/tr15/>

The Tokenization Pipeline



- A pre-tokenizer splits text into chunks. (often regex- or rule-based).
 - The subsequent tokenization (e.g., BPE) operates independently inside each chunk.

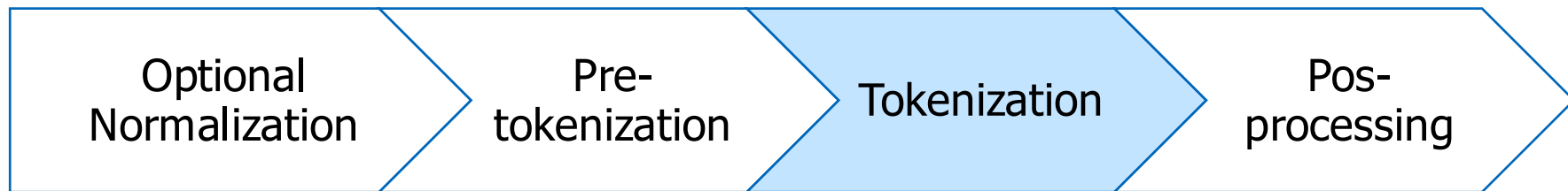
The cat saw a quokka 12345.

- Chunks:

[The] [·cat] [·saw] [·a] [·quokka] [·] [·] [123] [45] [·]

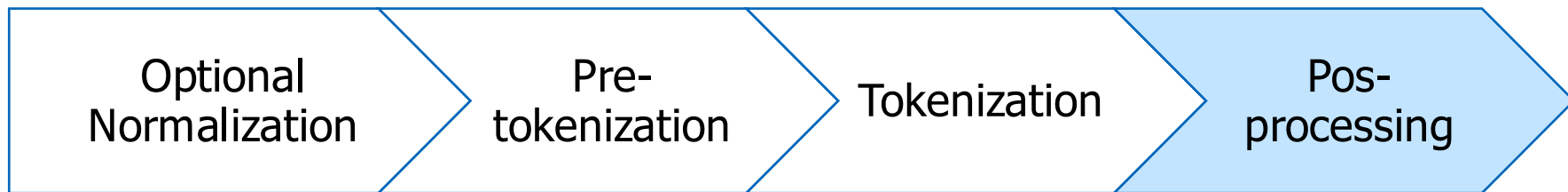
- Tokenization (e.g., BPE) can merge pieces within a chunk but cannot merge across pre-tokenization boundaries.

The Tokenization Pipeline



- **Training time:** Train the tokenizer on the pre-tokenized text (forthcoming)
- **Inference time:** Apply the trained tokenizer to each chunk.

The Tokenization Pipeline



- Truncate to match the maximum length of the model
- Pad all sentences in a batch to the same length
- Add special tokens: for example:
 - **<UNK>** (unknown word)
 - **<PAD>** (padding for fixed-length sequences)
 - **<BOS>** (beginning of sentence)
 - **<EOS>** (end of sentence)
- **Finally**, converts tokens into numerical IDs for model input.
 - Uses a vocabulary (word-to-index mapping).

Byte-pair Encoding (BPE) – An Example

- An algorithm for **forming subword** tokens based on **a collection of raw text**.

and there are no refueling stations anywhere
one of the city's more unprincipled real estate agents

Note that to do this tokenization, I need to learn it by seeing lots of text. (similar to model training)

*BPE-based
tokenization*

[and] [·there] [·are] [·no] [·re] [fueling] [·stations]
[·anywhere] [one] [·of] [·the] [·city] ['s] [·more] [·un]
[princi] [pled] [·real] [·estate] [·agents]

Byte-pair Encoding (BPE) Training

Overview:

- We are given a **large** text corpus of text.
 - Start by character-based tokenization.
 - Repeatedly merge the most frequent adjacent tokens
-
- Doing 30k merges => vocabulary of around 30k subwords. Includes many whole words.

Algorithm 1 Byte-pair encoding (Sennrich et al., 2016; Gage, 1994)

```
1: Input: set of strings  $D$ , target vocab size  $k$ 
2: procedure BPE( $D, k$ )
3:    $V \leftarrow$  all unique characters in  $D$ 
4:   (about 4,000 in English Wikipedia)
5:   while  $|V| < k$  do ▷ Merge tokens
6:      $t_L, t_R \leftarrow$  Most frequent bigram in  $D$ 
7:      $t_{\text{NEW}} \leftarrow t_L + t_R$  ▷ Make new token
8:      $V \leftarrow V + [t_{\text{NEW}}]$ 
9:     Replace each occurrence of  $t_L, t_R$  in
10:       $D$  with  $t_{\text{NEW}}$ 
11:   end while
12:   return  $V$ 
13: end procedure
```

BPE training: Example

- Form base vocabulary of all characters that occur in the training set.
- *Example:*

Our (very fascinating 🤖) training data: "jhu jhu jhu hopkins hop hops hops"

Base vocab: **h, i, j, k, n, o, p, s, u**

Tokenized data: j h u j h u j h u h o p k i n s h o p h o p s h o p s

Does not show the word separator for simplicity.

BPE training: Example (2)

- Count the frequency of each token pair in the data
- *Example:*

Our (very fascinating 🤖) training data: "jhu jhu jhu hopkins hop hops hops"

Base vocab: **h, i, j, k, n, o, p, s, u**

Tokenized data: j h u j h u j h u h o p k i n s h o p h o p s h o p s

Token pair frequencies:

- **j + h -> 3**
- **h + u -> 3**
- **h + o -> 4**
- **o + p -> 4**
- **p + k -> 1**
- **k + i -> 1**
-

BPE training: Example (3)

- Choose the pair that occurs more, merge them and add to vocab.
- *Example:*

Our (very fascinating 🤖) training data: "jhu jhu jhu hopkins hop hops hops"

Base vocab: h, i, j, k, n, o, p, s, u

Tokenized data: j h u j h u j h u h o p k i n s h o p h o p s h o p s

Token pair frequencies:

- j + h -> 3
- h + u -> 3
- h + o -> 4 ←
- o + p -> 4
- p + k -> 1
- k + i -> 1
-

BPE training: Example (4)

- Choose the pair that occurs more, merge them and add to vocab.
- *Example:*

Our (very fascinating 🤪) training data: "jhu jhu jhu hopkins hop hops hops"

Base vocab: h, i, j, k, n, o, p, s, u, ho ←

Tokenized data: j h u j h u j h u h o p k i n s h o p h o p s h o p s

Token pair frequencies:

- j + h -> 3
- h + u -> 3
- h + o -> 4 ←
- o + p -> 4
- p + k -> 1
- k + i -> 1
-

BPE training: Example (5)

- Retokenize the data
- *Example:*

Our (very fascinating 🤖) training data: "jhu jhu jhu hopkins hop hops hops"

Base vocab: h, i, j, k, n, o, p, s, u, ho

Tokenized data: j h u j h u j h u ho p k i n s ho p ho p s ho p s



Token pair frequencies:

BPE training: Example (6)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating 🤖) training data: "jhu jhu jhu hopkins hop hops hops"

Base vocab: **h, i, j, k, n, o, p, s, u, ho**

Tokenized data: j h u j h u j h u ho p k i n s ho p ho p s ho p s

Token pair frequencies:

- **j + h -> 3**
- **h + u -> 3**
- **ho + p -> 4**
- **p + k -> 1**
- **k + i -> 1**
- **i + n -> 1**
-

BPE training: Example (7)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating 🤖) training data: "jhu jhu jhu hopkins hop hops hops"

Base vocab: h, i, j, k, n, o, p, s, u, ho

Tokenized data: j h u j h u j h u ho p k i n s ho p ho p s ho p s

Token pair frequencies:

- j + h -> 3
- h + u -> 3
- ho + p -> 4 
- p + k -> 1
- k + i -> 1
- i + n -> 1
-

BPE training: Example (7)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating 🤪) training data: "jhu jhu jhu hopkins hop hops hops"

Base vocab: h, i, j, k, n, o, p, s, u, ho, hop ←

Tokenized data: j h u j h u j h u ho p k i n s ho p ho p s ho p s

Token pair frequencies:

- j + h -> 3
- h + u -> 3
- ho + p -> 4 ←
- p + k -> 1
- k + i -> 1
- i + n -> 1
-

BPE training: Example (7)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating 🤪) training data: "jhu jhu jhu hopkins hop hops hops"

Base vocab: h, i, j, k, n, o, p, s, u, ho, hop ←

Tokenized data: j h u j h u j h u hop k i n s hop hop s hop s ←

Token pair frequencies:

- j + h -> 3
- h + u -> 3
- ho + p -> 4 ←
- p + k -> 1
- k + i -> 1
- i + n -> 1
-

BPE training: Example (8)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating 🤖) training data: "jhu jhu jhu hopkins hop hops hops"

Base vocab: h, i, j, k, n, o, p, s, u, ho, hop

Tokenized data: j h u j h u j h u hop k i n s hop hop s hop s

Token pair frequencies:

- j + h -> 3 
- h + u -> 3
- hop + k -> 1
- hop + s -> 2
- k + i -> 1
- i + n -> 1
- n + s -> 1
-

BPE training: Example (8)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating 🤖) training data: "jhu jhu jhu hopkins hop hops hops"

Base vocab: h, i, j, k, n, o, p, s, u, ho, hop, jh ←

Tokenized data: j h u j h u j h u hop k i n s hop hop s hop s

Token pair frequencies:

- j + h -> 3 ←
- h + u -> 3
- hop + k -> 1
- hop + s -> 2
- k + i -> 1
- i + n -> 1
- n + s -> 1
-

BPE training: Example (8)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating 🤪) training data: "jhu jhu jhu hopkins hop hops hops"

Base vocab: h, i, j, k, n, o, p, s, u, ho, hop, jh ←

Tokenized data: jh u jh u jh u hop k i n s hop hop s hop s ←

Token pair frequencies:

- j + h -> 3 ←
- h + u -> 3
- hop + k -> 1
- hop + s -> 2
- k + i -> 1
- i + n -> 1
- n + s -> 1
-

BPE training: Example (8)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating 🤪) training data: "jhu jhu jhu hopkins hop hops hops"

Base vocab: h, i, j, k, n, o, p, s, u, ho, hop, jh

Tokenized data: jh u jh u jh u hop k i n s hop hop s hop s

Token pair frequencies:

- jh+u -> 3 
- hop + k -> 1
- hop + s -> 2
- k + i -> 1
- i + n -> 1
- n + s -> 1
-

BPE training: Example (8)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating 🤪) training data: "jhu jhu jhu hopkins hop hops hops"

Base vocab: h, i, j, k, n, o, p, s, u, ho, hop, jh, jhu ←

Tokenized data: jh u jh u jh u hop k i n s hop hop s hop s

Token pair frequencies:

- jh+u -> 3 ←
- hop + k -> 1
- hop + s -> 2
- k + i -> 1
- i + n -> 1
- n + s -> 1
-

BPE training: Example (8)

- Count the token pairs and merge the most frequent one
- *Example:*

Our (very fascinating 😊) training data: "jhu jhu jhu hopkins hop hops hops"

Base vocab: h, i, j, k, n, o, p, s, u, ho, hop, jh, jhu ←

Tokenized data: jhu jhu jhu hop k i n s hop hop s hop s ←

Token pair frequencies:

- j h+u -> 3 ←
- hop + k -> 1
- hop + s -> 2
- k + i -> 1
- i + n -> 1
- n + s -> 1
-

Limitations of BPE

- Loss of whole word semantics
 - E.g., “Understand” -> [“Under”, “stand”] -Doesn’t mean “stand beneath”!
- Language Dependency: Even though subwords helps in multiple-languages it may favor the structure of one language vs the other
 - Hard to apply to languages with **non-concatenative** (e.g., Arabic) morphology

كتب	k-t-b	“write” (root form)
كَتَبَ	kataba	“he wrote”
كَتَّبَ	kattaba	“he made (someone) write”
اِكتَتَبَ	iktataba	“he signed up”

Table 1: Non-concatenative morphology in Arabic.⁴
The root contains only consonants; when conjugating, vowels, and sometimes consonants, are interleaved with the root. The root is not separable from its inflection via any contiguous split.

Alternatives: WordPieces

- Merge by likelihood as measured by language model, not by frequency.

- While $\text{voc size} < \text{target}$:

1. Build a language model over your corpus
2. Merge tokens with the highest score:

$$\text{Score} = (\text{freq_of_pair}) / (\text{freq_of_first_element} \times \text{freq_of_second_element})$$

- **Rationale** — The algorithm prioritizes the merging of pairs where the individual parts are less frequent.
 - For example, ("un", "##able") may not merge despite its high frequency, as both parts appear frequently in corpus.
 - Conversely, ("hu", "##gging") may merge faster if “hugging” is common, as its parts are individually less frequent.

Alternatives: Unigram LM

- Unigram works in the other direction: it starts from a big vocabulary and removes tokens from it until it reaches the desired vocabulary size.
 - BPE/WordPieces are ‘bottom-up’ (merge characters).
 - Unigram is ‘top-down’ (prune substrings)

The algorithm computes how much the overall loss would increase if the symbol was removed and eliminates the symbols that would increase it the least.

Algorithm 2 Unigram LM (Kudo, 2018)

```
1: Input: set of strings  $D$ , target vocab size  $k$ 
2: procedure UNIGRAMLM( $D, k$ )
3:    $V \leftarrow$  all substrings occurring more than
4:     once in  $D$  (not crossing words)
5:   while  $|V| > k$  do ▷ Prune tokens
6:     Fit unigram LM  $\theta$  to  $D$ 
7:     for  $t \in V$  do ▷ Estimate token ‘loss’
8:        $L_t \leftarrow p_\theta(D) - p_{\theta'}(D)$ 
9:       where  $\theta'$  is the LM without token  $t$ 
10:    end for
11:    Remove  $\min(|V| - k, \lfloor \alpha |V| \rfloor)$  of the
12:    tokens  $t$  with highest  $L_t$  from  $V$ ,
13:    where  $\alpha \in [0, 1]$  is a hyperparameter
14:  end while
15:  Fit final unigram LM  $\theta$  to  $D$ 
16:  return  $V, \theta$ 
17: end procedure
```

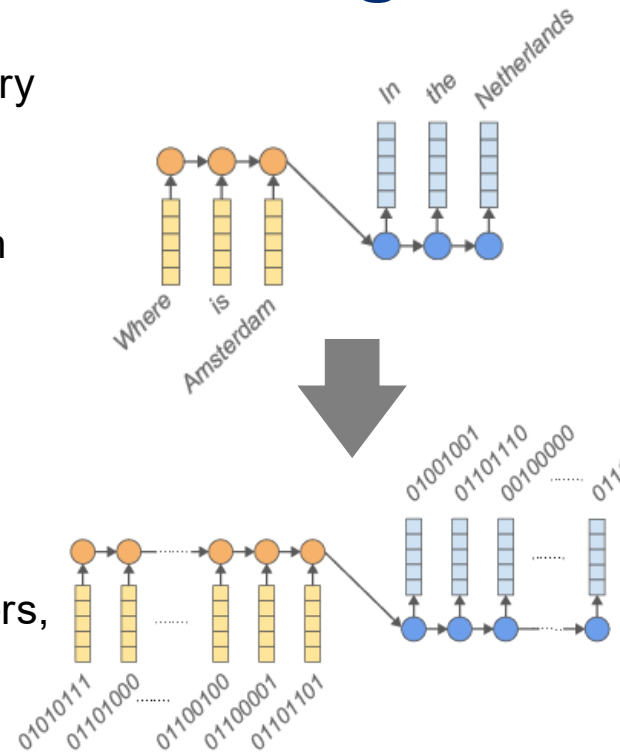
Alternatives: Unigram LM

Original:	furiously	Original:	tricycles	Original:	nanotechnology
BPE:	_fur iously	BPE:	_t ric y cles	BPE:	_n an ote chn ology
Uni. LM:	_fur ious ly	Uni. LM:	_tri cycle s	Uni. LM:	_nano technology
Original:	Completely preposterous suggestions				
BPE:	_Comple t ely _prep ost erous _suggest ions				
Unigram LM:	_Complete ly _pre post er ous _suggestion s				
Original:	corrupted		Original:	1848 and 1852,	
BPE:	_cor rupted		BPE:	_184 8 _and _185 2,	
Unigram LM:	_corrupt ed		Unigram LM:	_1848 _and _1852 ,	
Original	磁性は様々な分類がなされている。				
BPE	磁 性	は	様々	に分類	がなされている 。
Unigram LM	磁 性	は	様々	に 分類	がなされている 。
Gloss	magnetism	(top.)	various ways	in	classification is done .
Translation	Magnetism is classified in various ways.				

- Some (Bostrom and Durrett 2020) have argued that BPE produces less semantic tokens.
- But BPE based LMs do work fine – the transformer on top can do quite a bit.

Alternatives: Byte-level BPE Encoding

- Text gets encoded as one UTF-8 bytes, so the base vocabulary is the $2^8 = 256$ possible byte values.
 - E.g., H -> 01010111 (Byte value: 72)
 - Some works have used raw bytes as-is (e.g., ByT5) which leads to longer inputs.
- We train BPE on sequences of these bytes (i.e., merging frequently co-occurring byte pairs into new tokens).
- The final vocab (e.g., 50k tokens for GPT-2) is a mix of short byte seqs (for rare/unseen sequences) and longer byte-sequences (which end up corresponding to common characters, subwords, or whole words).
- Pros:** (1) no out-of-vocabulary problem ever; (2) no need for a huge base vocabulary.



Dealing with white spaces

- **Why Whitespace Isn't Just "Stripped Out"**
 - Whitespace are not stripped.
 - Goal: reversibility —tokenizers must reconstruct the original text, spacing included.
- **BPE:** Often attaches space as a *prefix* to the next word ("world" vs. " world" are **different** tokens);
 - GPT-2 maps space to a visible symbol "Ġ" for uniform byte-level processing.
- The issue w/ these is long spaces since they're never merged into longer tokens.
- GPT-4's fix (cl100k_base): introduced a new pre-tokenization regex and added dedicated multi-space tokens. This groups 4 spaces into a single token and has dedicated tokens for whitespace seqs up to 128 spaces.

GPT-2

```
def fibRec(n):↵
    if n < 2:↵
        return n↵
    else:↵
        return fibRec(n-1) + fibRec(n-2)
```

55 tokens

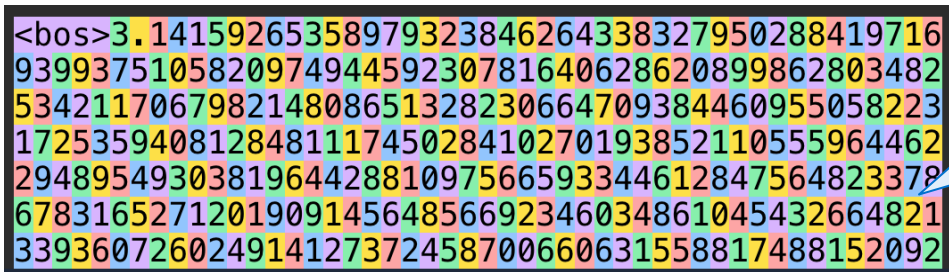
GPT-NeoX-20B

```
def fibRec(n):↵
    if n < 2:↵
        return n↵
    else:↵
        return fibRec(n-1) + fibRec(n-2)
```

39 tokens

Dealing with numbers

- Numbers are challenging:
 - BPE merges tokens based on frequency; digits frequency may be arbitrary.
 - Inconsistent tokenization depending on the context:
 - "1234" → 123+4, but "1235" → 12+35
 - "42" → 42 (single token; common token); "47" → 4+7
- **Fix #1:** Force every digit (0–9) to be its own token
 - Guarantees consistency: every "7" is always the same token
 - Trade-off: longer sequences for large numbers



<bos>3.14159265358979323846264338327950288419716939937510582097494459230781640628620899862803482534211706798214808651328230664709384460955058223172535940812848111745028410270193852110555964462294895493038196442881097566593344612847564823378678316527120190914564856692346034861045432664821339360726024914127372458700660631558817488152092

Mixtral, Llama, DeepSeek, and Gemma tokenizers broke down numerical sequences into a separate token for each digit.

Dealing with numbers

- Numbers are challenging:
 - BPE merges tokens based on frequency; digits frequency may be arbitrary.
 - Inconsistent tokenization depending on the context:
 - "1234" $\rightarrow$ 123+4, but "1235" $\rightarrow$ 12+35
 - "42" $\rightarrow$ 42 (single token; common token); "47" $\rightarrow$ 4+7
- Fix #1:** Force every digit (0–9) to be its own token
 - Guarantees consistency: every "7" is always the same token
 - Trade-off: longer sequences for large numbers
- Fix #2:** Let BPE do its merging on numbers, but cap its length (commonly 3).

3.141592653589793238462643383279502884197169399
375105820974944592307816406286208998628034825342
117067982148086513282306647093844609550582231725
359408128481117450284102701938521105559644622948
954930381964428810975665933446128475648233786783
165271201909145648566923460348610454326648213393
607260249141273724587006606315588174881520920962

GPT-4 and **GPT-4o** tokenizers broke down numerical sequences into groups of 3.

Dealing with numbers

- Numbers are challenging:
 - BPE merges tokens based on frequency; digits frequency may be arbitrary.
 - Inconsistent tokenization depending on the context:
 - "1234" → 123+4, but "1235" → 12+35
 - "42" → 42 (single token; common token); "47" → 4+7
- **Fix #1:** Force every digit (0–9) to be its own token
 - Guarantees consistency: every "7" is always the same token
 - Trade-off: longer sequences for large numbers
- **Fix #2:** Let BPE do its merging on numbers, but cap its length (commonly 3).
- **Fix #3:** Right-to-left chunking are shown to be more effective.
 - Chunking digits from the left doesn't align with base-10 place value
 - "1234567" → 123 | 456 | 7 (left; misaligned) vs 1 | 234 | 567

Special tokens

- Special tokens are **manually** inserted into the vocab by model developers.
- These are **not** meant to be used or shown by human users. They're parsed by the system that is running the LM. Hence, they use strings that're unlikely to occur naturally.
- Examples from GPT-OSS:
 - `<|start|>` / `<|end|>` : mark the **start/end of a message**
 - `<|message|>` : separates a message's **header (role) from its content**
 - `<|channel|>` : labels **which output stream** a piece of text belongs to (reasoning vs. final answer)
 - `<|constrain|>` : specifies the **expected format for tool-call arguments**
 - `<|call|>` : marks the model **invoking a tool**
 - `<|return|>` : marks the **final end of a response**

See OpenAI's special tokens:

https://github.com/openai/tiktoken/blob/08a5f3b2c987ada4fc5aa1f16c643c203fa8acaa/tiktoken_ext/openai_public.py#L123-L144

<https://developers.openai.com/cookbook/articles/openai-harmony#special-tokens>

Special tokens

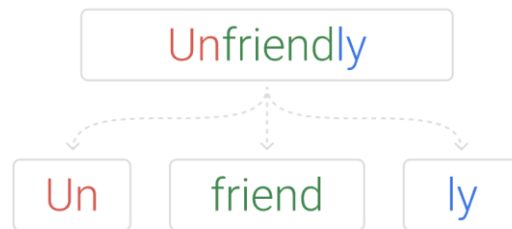
- Most tokenizers reserve a block of IDs up front for special tokens (e.g., Qwen reserves 208 extra empty slots).
- So new ones can be added later without resizing the whole vocabulary/embedding matrix.

Model	Release	Reserved block
Kimi K2, K2.5, K2.6	2025-26	3840
Qwen 2.5/3	2024-25	208
Gemma 3	2025	99
Llama 3	2024	256

Summary

- **Question:** what is a good atomic unit for feeding lang to LMs?

- **Words:** too coarse
- **Characters:** too small
- **Subwords:** a good middle ground.



- **One nice intuition:** tokenization is compression for [almost] free.

What Tokenizers do people use?

Model	Release	Algorithm	Library
Kimi K2, K2.5, K2.6	2025-26	Byte-level BPE	tiktokens
GLM-4/GLM-4.6	2024-25	Byte-level BPE	HF tokenizers
GPT-OSS	2025	Byte-level BPE	tiktokens
Qwen 2.5/3	2024-25	Byte-level BPE	tiktokens
Gemma 3	2025	Unigram LM	SentencePieces
DeepSeek-V3	2024	Byte-level BPE	HF tokenizers

- Nearly everyone (except Google) uses BPE.
- All these tokenizers are *mostly** *invertible* (*except the ones that do aggressive normalization)

Vocab sizes

Model	Release	Vocab size
Kimi K2, K2.5, K2.6	2025-26	160k
GLM-4/GLM-4.6	2024-25	150k
GPT-OSS	2025	201k
Qwen 2.5/3	2024-25	151k
Gemma 3	2025	262k
DeepSeek-V3	2024	128k
mT5	2020	250k

Model	Release	Vocab size
Original Transformer	2017	37k
GPT1	2018	40k
GPT2/3	2019-20	50.2k
T5	2019	32k
Llama	2023	32k
GPT4	2023	100.2k

- Vocab size of English-only models (**right**) had 30-60k.
- While multi-lingual models (**left**) typically have >> 100k tokens.
- Some teams size vocabs to align with hardware/tensor-parallel efficiency.
 - $2^{15} = 32,768$; $2^{16} = 65,536$; $131,072 = 2^{17}$; $262,144 = 2^{18}$;

Summary

- Everyone uses invertible subword tokenizers (BPE, Unigram) for good reason.
- What are some new ideas in tokenization?

SuperBPE

- The standard BPE first splits the words into chunks separated by white spaces. So that a token in the vocab can *never cross the white space border*.
- What if tokens contained whitespace?
 - Multi-word expressions act as single semantic units (e.g., "by the way", "for example")
 - Longer tokens means model can generate more efficiently.
- **Approach:**
 - **Stage 1:** learn ordinary subwords (like normal BPE)
 - **Stage 2:** learn "superwords" that span multiple words/whitespace
- **Critique:** Only works for languages for many white spaces (e.g. English). What about Chinese, Japanese and other no-white-space languages?

SuperBPE

POS tag	#	Example Tokens
NN, IN	906	_case_of, _hint_of, _availability_of, _emphasis_on, _distinction_between
VB, DT	566	_reached_a, _discovered_the, _identify_the, _becomes_a, _issued_a
DT, NN	498	_this_month, _no_idea, _the_earth, _the_maximum, _this_stuff
IN, NN	406	_on_top, _by_accident, _in_effect, _for_lunch, _in_front
IN, DT	379	_on_the, _without_a, _alongside_the, _for_each
IN, DT, NN	333	_for_a_living, _by_the_way, _into_the_future, _in_the_midst
NN, IN, DT	270	_position_of_the, _component_of_the, _review_of_the, _example_of_this
IN, DT, JJ	145	_like_any_other, _with_each_other, _for_a_short, _of_the_entire
VB, IN, DT	121	_worked_as_a, _based_on_the, _combined_with_the, _turned_into_a
IN, DT, NN, IN	33	_at_the_time_of, _in_the_presence_of, _in_the_middle_of, _in_a_way_that
,, CC, PRP, VB	20	, _and_it_was, , _but_I_think, , _but_I_have, , _but_I_am
IN, DT, JJ, NN	18	_in_the_long_run, _on_the_other_hand, _for_the_first_time, _in_the_same_way

Table 3: **The most common POS tags for tokens of 2, 3, and 4 words in SuperBPE**, along with random example tokens for each tag. NN = noun, IN = preposition, VB = verb, DT = determiner, CC = conjunction, JJ = adjective, PRP = pronoun.

Lang Tokenization in Visual Space

- Some prior work have tried encoding text as pixels. (Rust et al.)
- Recent works (e.g., DeepSeek-OCR) render a page as an image, and encode it into a small number of "vision tokens".
- They boast 10x compression with minimal accuracy drop.

